

编译原理第一、三章作业参考答案

1 第一次作业

练习1.1.1

（答案不唯一）编译器的主要优点是：程序经过翻译后可直接在目标平台运行，通常执行效率较高；编译阶段能够进行较充分的优化和静态错误检查；可脱离源程序发布目标代码。解释器的主要优点是：便于交互式执行和调试；程序修改后可立即运行；通常具有较好的可移植性，因为只需为不同平台实现解释器即可。

练习1.1.2

（答案不唯一）编译器产生汇编语言而不是机器语言的好处包括：汇编语言可读性较好，便于调试和检查编译结果；可利用现有汇编器完成指令编码、重定位和目标文件生成；编译器后端实现更简单，也更容易移植到已有汇编工具链支持的平台。

练习1.1.3

(1)

$$w = 24, \quad x = 91, \quad y = 169, \quad z = 20.$$

(2)

$$w = 16, \quad x = 2, \quad y = 28, \quad z = 18.$$

练习1.1.4

宏定义 `#define a x` 在预处理阶段把标识符 `a` 替换为 `x`。函数 `b` 中的 `x=a*3` 等价于 `x=x*3`，修改全局变量 `x`，由 4 变为 12，因此第一次打印 12。函数 `c` 中声明了局部变量 `int x=1111`，宏 `a+1` 展开为 `x+1`，此处的 `x` 指局部变量，因此第二次打印 1112。

练习1.1.5*

(答案不唯一，充分讨论即可) 我个人认为把语言简单划分为“编译型语言”和“解释型语言”并不合理。编译和解释是语言实现方式，同一种语言既可以有编译实现，也可以有解释实现。例如，可以构造 C 语言解释器，直接解释执行 C 程序或其中间表示；也可以把 Python 程序静态编译成本地代码。还有多种机制混合，例如先编译到字节码(如Java)，再由虚拟机解释执行或即时编译执行。

2 第三次作业

练习3.1.1

程序如下：

```
int sum(int n){
    /* compute prefix sum */
    return n<=0 ? 0 : n + sum(n-1);
}
```

注释不产生词法记号。词素序列如下：

```
<int>
<id, 指向 sum 的符号表项指针>
<op, "(">
<int>
<id, 指向 n 的符号表项指针>
<op, ")">
<op, "{">
<return>
```

```

<id, 指向 n 的符号表项指针>
<op, "<=">
<num, 0>
<op, "?">
<num, 0>
<op, ":">
<id, 指向 n 的符号表项指针>
<op, "+">
<id, 指向 sum 的符号表项指针>
<op, "(">
<id, 指向 n 的符号表项指针>
<op, "-">
<num, 1>
<op, ")">
<op, ";">
<op, "}">

```

答案将运算符和界符统一记为 `op`，可以将其分别记为 `rel-op`、`add-op`、`delimiter` 等。言之有理即可。

练习3.1.2

(1) $b(a|b)^*b$:

所有由 a, b 组成、以 b 开头且以 b 结尾的串，长度至少为 2。

(2) $(a|ba)^*$:

由若干个 a 或 ba 连接而成的串。或：串中每个 b 后面都必须紧跟一个 a ，因此串不能以 b 结尾，也不能出现连续的 bb 。

(3) $(a|b)^*b(a|b)(a|b)$:

所有由 a, b 组成、倒数第三个字符为 b 的串，长度至少为 3。

练习3.1.3

(1) 包含5个元音的所有小写字母串，这些串中的元音按顺序出现

设 C 表示非元音小写字母集合，即

$$C = (b|c|d|f|g|h|j|k|l|m|n|p|q|r|s|t|v|w|x|y|z).$$

若要求恰好包含五个元音且顺序为 a, e, i, o, u ，则正则表达式为

$$C^* a C^* e C^* i C^* o C^* u C^*.$$

(2) 所有由a和b组成且不含子串baa的串

所有由 a, b 组成且不含子串 baa 的串可表示为

$$a^* (b|ba)^*.$$

(3) 所有由a和b组成且不含子序列baa的串

所有由 a, b 组成且不含子序列 baa 的串可表示为

$$a^* b^* (\epsilon | ab^*).$$

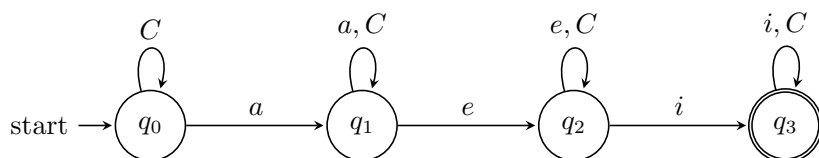
注意子序列 ($b...a...a$) 和子串 (baa) 的区别。

3 第四次作业

练习3.2.1

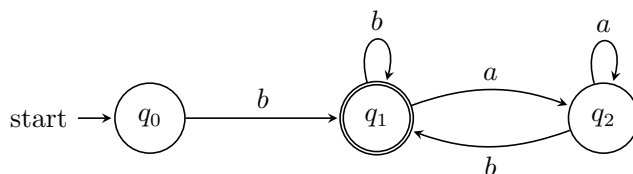
(1) 所有由小写字母组成、且 3 个元音 a, e, i 必须按此顺序至少各出现一次的串

设 Σ 为全体小写字母， $C = \Sigma \setminus \{a, e, i\}$ 。



(2) 所有由 a 和 b 构成、以 b 开头并以 b 结尾的串

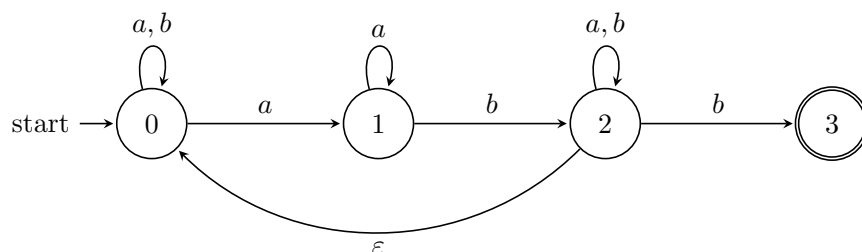
正则表达式: $b(a|b)^*b|b$ 。| 前的表达式最少匹配到 bb , 加上 $|b$ 是为了覆盖单独一个 b 的情形。



练习3.2.2

NFA 转移:

$$\begin{aligned} 0 &\xrightarrow{a,b} 0, & 0 &\xrightarrow{a} 1, & 1 &\xrightarrow{a} 1, & 1 &\xrightarrow{b} 2, \\ 2 &\xrightarrow{a,b} 2, & 2 &\xrightarrow{b} 3, & 2 &\xrightarrow{\varepsilon} 0. \end{aligned}$$



ε -closure 计算:

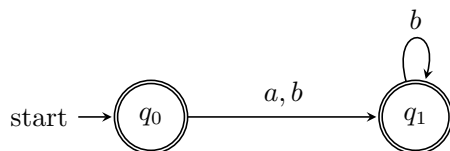
$$\varepsilon\text{-closure}(0) = \{0\}, \quad \varepsilon\text{-closure}(1) = \{1\}, \quad \varepsilon\text{-closure}(2) = \{0, 2\}, \quad \varepsilon\text{-closure}(3) = \{3\}.$$

Input	move	ε -closure
ε	—	$\{0\}$
a	$\{0, 1\}$	$\{0, 1\}$
ab	$\{0, 2\}$	$\{0, 2\}$
abb	$\{0, 2, 3\}$	$\{0, 2, 3\}$
$abba$	$\{0, 1, 2\}$	$\{0, 1, 2\}$

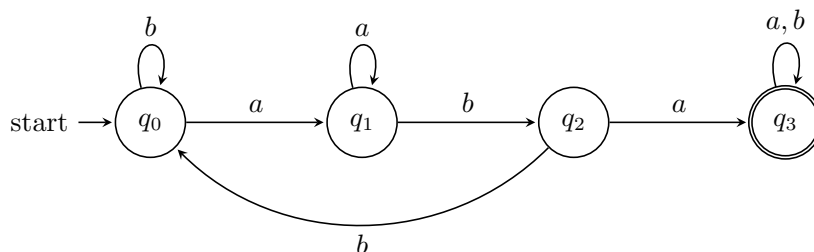
最后一步: 从 $\{0, 2, 3\}$ 读入 a , $0 \rightarrow \{0, 1\}$, $2 \rightarrow \{2\}$, 3 在 a 上无定义, 合并得 $\{0, 1, 2\}$; 其 ε -闭包仍为 $\{0, 1, 2\}$ 。最终状态集合不含接受态 3 , 故 NFA 不接受 $abba$ 。

练习3.2.3

(1) $(a|\varepsilon)b^*$



(2) $(a|b)^*aba(a|b)^*$



练习3.2.4 二进制串中能被 5 整除的正整数

结论：能。该语言是正则的，因此可由正则表达式描述。

核心思想：模 5 的余数只有 5 种取值。设当前已读入二进制串对应的值为 v ，余数为 $r = v \bmod 5$ 。再读入一位 $b \in \{0, 1\}$ 后值为 $v' = 2v + b$ ，故新余数为

$$r' = (2r + b) \bmod 5.$$

DFA 构造：5 个状态 q_0, q_1, q_2, q_3, q_4 ，其中 q_r 表示当前余数为 r 。转移函数 $\delta(q_r, b) = q_{(2r+b) \bmod 5}$ ，得到：

	0	1
q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_4	q_0
q_3	q_1	q_2
q_4	q_3	q_4

为了排除空串与首位零（题设要求是正整数），增加一个起始状态 s ： $s \xrightarrow{1} q_1$ 。可以找一些串自行验证：

• $101_2 = 5$: $s \xrightarrow{1} q_1 \xrightarrow{0} q_2 \xrightarrow{1} q_0$, 接受 ✓

• $11_2 = 3$: $s \xrightarrow{1} q_1 \xrightarrow{1} q_3$, 拒绝 ✓

由于该语言能被 DFA 识别, 为正则语言, 存在与之等价的正则表达式。

对 $q_0, \dots, q_4$ 消去 q_4, q_3, q_2, q_1 。令

$$A = 10 \mid (0 \mid 11)(01^*01)^*01^*00,$$

$$B = (0 \mid 11)(01^*01)^*1,$$

得到所有值能被 5 整除的二进制串 (含零, 还不是正整数) 的正则表达式

$$(0 \mid 1A^*B)^*.$$

题设要求正整数 (首位强制为 1), 得到:

$$1 \cdot A^*B \cdot (0 \mid 1A^*B)^*.$$

